SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 11)

TASK 11 – RECOMMEND AND DEFEND THE OPERATING-SYSTEM PLAN

11.1 Complete recommended plan

Hospital component	OS type / structure	Services, protection and I/O
Bedside monitors	Embedded hard-RTOS; microkernel-style where validated, or a minimal RTOS with a documented isolation strategy.	Fixed-priority scheduling, bounded interrupt handling, local alarm path, watchdog, secure boot/update, sensor interrupt/DMA as supported, minimum services and local buffering.
Nurses' tablets	Managed handheld/mobile OS.	Power management, encrypted storage, user authentication, app allow-list, secure Wi-Fi/TLS, remote lock/wipe and controlled updates.
Doctors' workstations	Desktop OS; centrally managed.	Clinical applications, endpoint protection, user-mode isolation, disk encryption, MFA where available, RBAC and audited access.
Patient database	Clustered server OS on separate nodes; modular Linux kernel is a practical option.	Database service accounts, RBAC, encryption, replication, quorum/fencing, health checks, backups, audit logs and fail-over testing.
MRI imaging	SMP server OS with parallel processing and high-throughput storage.	Multi-core scheduling, large RAM, DMA, NVMe/SSD, separate storage/network permissions and workload monitoring.
Partner-hospital link	Distributed-system interfaces through secure gateways.	Authenticated APIs, encrypted transport, certificates, network segmentation, least-privilege exchange, retries and audit trails.

11.2 Why this combination is stronger than one OS

The design matches the OS to the dominant risk of each environment. The monitor receives deterministic scheduling and a small trusted code base. Tablets receive mobile power management. Workstations receive broad interactive application support. The database receives cluster fail-over and strong data protection. The imaging host receives SMP and high-throughput I/O. Distributed services connect the hospital group without making a remote network part of the local alarm loop.

11.3 Limitations and risks

- Operating-system diversity increases patching, staff training, compatibility testing and configuration-management effort. Central policy and automation are needed to manage versions and security baselines.
- A microkernel or RTOS does not guarantee safety automatically. The exact hardware, drivers, interrupt path, scheduling priorities and worst-case execution times must be validated.
- A database cluster can fail if replication, quorum, fencing or storage design is incorrect. Split-brain and inconsistent backups are major risks; recovery drills are required.
- Network segmentation and encryption add configuration complexity. Certificates, time synchronization, access policies and key rotation must be maintained.
- A software update can introduce regressions. Stage updates, verify signatures, keep rollback images and test them before deploying to clinical devices.
- A distributed service can be unavailable during WAN failure. Each hospital needs a documented degraded mode and a safe approach for delayed synchronization.

11.4 Suitability as the hospital grows from one ward to three hospitals

Scale	Design response
One ward: 40 monitors, 10 tablets	A local ward gateway collects readings; monitor alarms remain local. A small local service can relay telemetry to hospital systems.

Scale	Design response
One hospital: 500 monitors, 120 tablets, 80 workstations	Use segmented clinical networks, redundant gateways, central identity management, clustered patient records, a separate imaging server and monitored wireless infrastructure.
Three hospitals sharing records	Adopt a distributed architecture with secure inter-site APIs, replicated/cloud record services, site-level network gateways, identity federation and clear data ownership. Keep local critical services available if WAN connectivity is interrupted.

11.5 Alternative: move everything into virtual machines on two large servers

Virtual machines can consolidate desktops, application servers, database services and some test workloads. They improve deployment flexibility and can support snapshots and migration. However, placing the bedside alarm path into ordinary VMs on two large shared servers is not recommended. Hypervisor scheduling, shared CPU contention, device virtualization and host failure can add latency and jitter. The 10 ms alarm should remain on dedicated embedded hardware with a validated real-time OS.

Virtualization also requires memory for the hypervisor, guest OS kernels and virtual devices, plus CPU overhead for scheduling, I/O virtualization and emulation. Two physical servers create large failure domains: a host, storage or network failure may affect many virtual machines simultaneously. Redundant hosts, independent storage, power, network paths and tested fail-over are required. Use virtualization for suitable server workloads, but do not consolidate away the isolation and availability needed by the monitors and database.

Final judgement: use a hybrid physical/virtual environment. Keep patient monitors physically independent; use clustered and possibly virtualized server infrastructure where latency and availability requirements are proven; and maintain redundancy beyond only two machines